Git Repository Structure — FRE 5040

This document is the source of truth for where every file goes. If any other handout, notebook, or slide shows something different, this document wins — tell the instructor so it can be corrected.

Keep it open while you work. Almost every question of the form *"where does this go?"* is answered here.

---

1. The three places your work lives

This is the single most important idea in this document. Your repository has **three separate areas**, and they never mix.

Area	What it is	Pushed to GitHub?
`class_materials/`	Handouts from the instructor. Read them, run the lecture notebooks here.	**No** — gitignored
`homework/`	One folder per stage. The week's exercise, self-contained.	**Yes**
`project/`	Your cumulative course project. Grows all semester.	**Yes**

Why `class_materials/` is not pushed. It holds files the instructor gave everyone, plus whatever the lecture notebooks generate while you follow along. Nobody needs 160 copies of the same handout in 160 repositories, and the generated files are scratch. Because it is gitignored, you can always re-run a lecture notebook from a clean copy without polluting your repo.

Homework and project are two deliverables, not one

- **Homework** is the week's exercise. You do it on the data we give you, in `homework/homeworkNN/`. It stands alone.
- **The project** is your own dataset, carried across the whole semester, in `project/`.

They deliberately overlap. You will often do something in homework, then adapt it for your project. **Adapting is the point** — it is not copying. The homework sheet governs the homework; the project instructions govern the project.

---

2. The complete structure — folders only

This is what your repository looks like by the **end of the course**.

```
bootcamp_<firstname>_<lastname>/
│
├── class_materials/            ← handouts; gitignored, never pushed
│   ├── stage00_preclass-setup/
│   ├── stage01_problem-framing-and-scoping/
│   ├── stage02_tooling-setup/
```

```
│   └── …                       one folder per stage, full stage name
│
├── homework/                   ← each folder holds ONLY what that week uses
│   ├── homework00/                 setup only — no data folders
│   ├── homework01/                 problem framing — no data folders
│   ├── homework02/                 the full structure, built once as practice
│   ├── homework03/
│   │   ├── data/raw/
│   │   ├── data/processed/
│   │   └── src/
│   ├── homework04/
│   │   └── data/raw/
│   ├── homework05/
│   │   ├── data/raw/
│   │   └── data/processed/
│   ├── homework06/
│   │   ├── data/raw/
│   │   ├── data/processed/
│   │   └── src/
│   ├── homework07/
│   │   ├── data/raw/
│   │   ├── data/processed/
│   │   └── src/
│   ├── homework08/                 notebook only
│   ├── homework09/
│   │   └── src/
│   ├── homework10a/                notebook only
│   ├── homework10b/                notebook only
│   ├── homework11/
│   │   ├── data/raw/
│   │   ├── data/processed/
│   │   └── src/
│   ├── homework12/
│   │   ├── data/processed/
│   │   └── reports/
│   │       └── images/
│   └── …                       stages 13–16
│
├── project/
│   ├── data/
│   │   ├── raw/                ← direct and unedited from the source
│   │   └── processed/          ← derived tables, reproducible from your code
│   ├── notebooks/
│   ├── src/
│   ├── reports/
│   │   └── images/
│   ├── model/
│   ├── docs/
│   └── README.md
│
├── .gitignore
└── README.md
```

Homework folder numbers match stage numbers, zero-padded: stage 04 → `homework04`. Stage 10 is split into two, so its homework folders are `homework10a` and `homework10b`.

What each project folder is for

Folder	Holds
`data/raw/`	Your inputs, direct and unedited from the source. To fix something, write code that reads from `raw/` and writes to `processed/`.
`data/processed/`	Anything derived from raw data by your code. Deletable and re-creatable.
`notebooks/`	Your project notebooks.
`src/`	Reusable functions you import — the modules you extract from notebooks.
`reports/`	Deliverables for a reader: summaries, charts, PDFs.
`reports/images/`	Figures saved by your code.
`model/`	Saved model objects.
`docs/`	Internal design notes — memos, personas, assumptions.

Nothing sits loose in `data/`. It contains only `raw/` and `processed/`. If a stage needs just one of them, use just that one.

---

3. The complete structure — with files

The same tree, showing the files that exist by the end. Names marked `<...>` vary with your work.

```
bootcamp_<firstname>_<lastname>/
│
├── class_materials/
│   └── stage04_data-acquisition-and-ingestion/
│       ├── stage04_…_lecture-notebook.ipynb      ← open and run HERE
│       ├── stage04_…_reading-text.md
│       ├── stage04_…_homework-sheet.md
│       ├── stage04_…_homework-starter.ipynb      ← copy OUT to homework04/
│       ├── .env.example
│       └── data/raw/                              ← created when you run the
notebook
│           └── <generated files>                     timestamped; scratch, not
pushed
│
├── homework/
│   └── homework04/
│       ├── homework04_data-acquisition-and-ingestion_submission.ipynb
│       └── data/raw/
│           └── <the CSVs your notebook saved>
│
├── project/
│   ├── data/
```

```
│   │   ├── raw/          <your acquired data>
│   │   └── processed/    <your cleaned/derived data>
│   ├── notebooks/
│   │   └── project_pipeline.ipynb    ← started in stage 04, extended every stage
after
│   ├── src/
│   │   ├── config.py
│   │   ├── utils.py
│   │   ├── cleaning.py
│   │   ├── outliers.py
│   │   ├── features.py
│   │   └── evaluation.py
│   ├── reports/
│   │   ├── images/       <your figures>
│   │   └── <your summaries and deliverables>
│   ├── model/            <your saved models>
│   ├── docs/             <your design notes>
│   ├── requirements.txt
│   ├── .env.example      ← committed: the template, no real keys
│   ├── .env              ← NOT committed: your real keys
│   └── README.md
│
├── .gitignore
└── README.md
```

You build all of `project/` at once, in stage 02. Its shape is the same for every student and never changes, so you create it complete and then fill it as the course proceeds. Folders that are still empty need a `.gitkeep` file — see *Git will not commit an empty folder*, below.

---

4. Homework folders hold only what that week needs

Each homework folder uses the **same vocabulary** as the project — `data/raw/`, `src/` — but only the folders that week actually uses. Do not create the rest.

Homework	Folders it uses
`homework03`	`data/raw/` `data/processed/` `src/`
`homework04`	`data/raw/`
`homework05`	`data/raw/` `data/processed/`
`homework06`	`data/raw/` `data/processed/` `src/`
`homework07`	`data/raw/` `data/processed/` `src/`
`homework08`	*(notebook only)*
`homework09`	`src/`
`homework10a`	*(notebook only)*
`homework10b`	*(notebook only)*

Homework	Folders it uses
homework11	data/raw/ data/processed/ src/
homework12	data/processed/ reports/ reports/images/
homework13–homework16	…

Three stages work differently, and it is deliberate:

- **Stage 00** — homework00 holds your setup work: python_tutorial.ipynb and the repository itself. It is about getting the machine ready, not about data.
- **Stages 01 and 02** — the *subject* of these two is setting up: framing the problem, and building the environment and repository. You still do the work in homework01/ and homework02/ like any other week; what carries into project/ is the framing itself and the working scaffold, rather than an analysis. Homework 02 is the one place you build **all seven folders** in a homework folder — deliberately, as practice, before you build the real ones in project/.

Your homework notebook sits at the root of the homework folder, not inside a notebooks/ subfolder — so that src/ sits beside it and data/raw/ lands where the homework sheet says.

Name it to match the stage:

```
homework/homework03/homework03_python-fundamentals_submission.ipynb
```

homework<NN> · the stage name · _submission · .ipynb. The homework prefix keeps your work visibly distinct from the stage… files we hand you.

---

5. Run this cell first in your project notebook

Paste this at the top of project/notebooks/project_pipeline.ipynb:

```
# --- run me first: makes this notebook work wherever it lives ---
from pathlib import Path
import os, sys
if Path.cwd().name == 'notebooks':
    os.chdir('..')                    # project/notebooks -> project
ROOT = Path.cwd()
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))     # so `from src.… import …` works
print('working from:', ROOT.name)
```

You do not need this in your homework notebooks. Jupyter starts a notebook in its own folder. A homework notebook sits at the top of homework/homeworkNN/, right beside its own src/ and data/, so imports and paths already work — the cell would do nothing there.

Your project notebook is the one exception, because it sits one level down inside `notebooks/`. Without the cell:

```
from src.cleaning import clean   ->  ModuleNotFoundError: No module named 'src'
data/raw/prices.csv              ->  written to project/notebooks/data/raw/
(wrong)
```

The cell moves you up to `project/`, so `src/` and `data/` resolve the way the rest of these instructions describe. It is safe to run more than once.

Never write `../data/` — in homework you do not need it, and in the project this cell removes the need. If you find yourself typing `..`, something is wrong with where you are working, not with the path.

---

6. Git will not commit an empty folder

This surprises everyone, so read it twice: **git tracks files, not folders.** If you create `reports/` and push before putting anything in it, the folder simply will not appear on GitHub. Nothing is broken — there is nothing to record.

Two ways to handle it, and the course uses both:

- **Homework** — don't create a folder until you have something to put in it. That is the rule in *Homework folders hold only what that week needs*, above.
- **Project** — you build the whole tree in stage 02, so most of it starts empty. Create an empty file named `.gitkeep` inside each folder that has nothing in it yet, then commit as usual.

`.gitkeep` is not a git feature — it is a convention. Any file would do; this name tells the next reader why it is there. Delete it once the folder has real content, or leave it; both are fine.

---

7. What is and is not pushed

`.gitignore`, given to you in stage 00:

```
# Local class materials — never pushed
class_materials/

# Secrets
.env

# Python
__pycache__/
*.pyc
*.pyo

# Jupyter
.ipynb_checkpoints/
```

```
# OS
.DS_Store
```

`data/` **is committed.** Several stages grade the files you save into `data/raw/` and `data/processed/`; your GA cannot grade what was never pushed. Keep the files small.

`.env` **is never committed.** `.env.example` **always is.** `.env` holds your real API keys; `.env.example` is the same file with the values replaced by placeholders, so a classmate knows which keys they need. Copy `.env.example` to `.env` and fill in your own values.

---

8. Writing paths

No leading slash. Write `data/raw/prices.csv`, never `/data/raw/prices.csv`. A leading slash means the root of your whole hard drive, not your project — on macOS and Linux it fails outright.

Forward slashes, even on Windows. Python accepts them everywhere; backslashes are escape characters and cause bugs.

Better still, build paths from `ROOT` (defined by the setup cell above):

```
RAW = ROOT / 'data' / 'raw'
df.to_csv(RAW / 'prices.csv', index=False)
```

---

9. Common questions

Does my notebook go in the homework folder or the project? You have one notebook per homework, and **one** notebook for the whole project. The week's exercise goes in `homework/homeworkNN/` — a new notebook each week. The project has a single notebook, `project/notebooks/project_pipeline.ipynb`, which you start in stage 04 and add to every stage after. Adapting your homework work into it is the assignment, not a shortcut.

Do all my homeworks go in one folder? No. One folder per stage: `homework/homework03/`, `homework/homework04/`, and so on.

I created the folders and pushed, but GitHub shows nothing. The folders were empty. See *Git will not commit an empty folder*, above.

Where do I put the file the instructor gave me? In `class_materials/stage<NN>_<full-stage-name>/`, matching the stage folder name exactly. Run lecture notebooks from there. To do the homework, **copy** the starter into your homework folder and work on the copy — never edit anything inside `class_materials/`, so you always keep a clean original.

If a handout shows a different folder layout. This document wins. Please tell the instructor.